

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

7. Hive, Impala y Sqoop	1
7.1. Hive	1
7.1.1. Arquitectura	1
7.1.2. Base de Datos columnar	3
7.1.3. Particiones	4
7.1.4. Metadatos	4
7.1.5. Ingesta de Datos	4
7.1.6. SQL	5
7.2. Impala	6
7.2.1. Visión	6
7.2.2. Arquitectura	6
7.2.3. Soporte SQL	9
7.2.4. Almacenamiento HDFS	9
7.3. Sqoop	10
7.3.1. Conectores	11
7.3.2. Importación/Exportación	12
7.3.3. Comandos Sqoop	13

Listado de acrónimos

API	Application Programming Interface
HDFS	Hadoop Distributed File System
LLAP	Live Long And Process
MPP	Multiple Parallel Processing
ORC	Optimized Row Columnar
RDBMS	Relational Database Management System

7

Capítulo

Hive, Impala y Sqoop

Francisco P. Romero

7.1. Hive

Apache Hive es un software, implementado en Java, de almacenamiento de datos que facilita la lectura, escritura y gestión de grandes conjuntos de datos localizados en un almacenamiento distribuido utilizando SQL [SVGK16]. Hive permite el almacenamiento de datos en el ecosistema Apache Hadoop. Puede ejecutarse en clusters tradicionales de Hadoop o en entornos en la nube. Inicialmente, Hive se utilizaba principalmente para ETL y el procesamiento por lotes. Aunque sigue soportando estos casos de uso, ha evolucionado para soportar también casos de uso de data warehousing como la elaboración de informes, consultas interactivas e inteligencia empresarial. Esta evolución se ha llevado a cabo mediante la adopción de diferentes técnicas comunes de almacenamiento de datos y adaptando esas técnicas al al ecosistema Hadoop.

7.1.1. Arquitectura

La arquitectura de Hive se muestra en la figura 7.2. No todos de los componentes del diagrama son necesarios en todas las instalaciones. LLAP y HiveServer2 son opcionales; el Metastore puede ejecutarse incrustado en HiveServer2 o en el código del cliente.



Hive utiliza el término esquema para referirse a los nombres de las columnas y los tipos de datos de una relación, también conocido como esquema de la tabla. En un contexto RDBMS esquema a menudo se refiere a una colección de relaciones. A lo largo de este artículo, el término esquema se refiere a un esquema de tabla a menos que se especifique lo contrario.

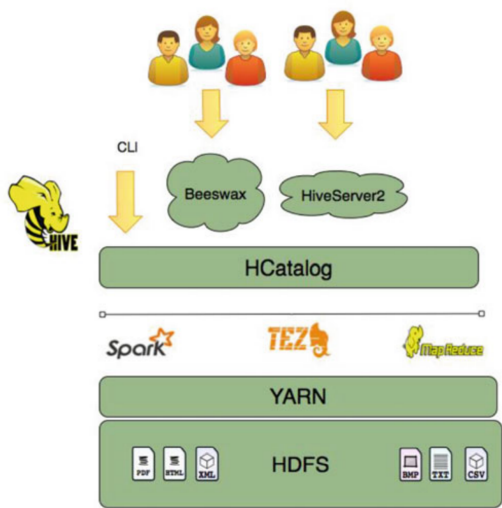


Figura 7.1: Vision Global de Hive. Fuente:[SVGK16]

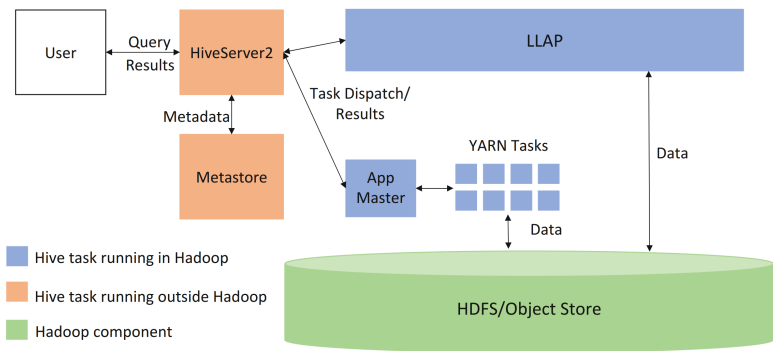


Figura 7.2: Arquitectura de Hive. Fuente:Apache

La ejecución sobre Hadoop da a Hive la capacidad de ejecutarse sobre grandes conjuntos de datos con un alto grado de paralelismo. Además, dado que Hadoop proporciona conectores a otros sistemas de almacenamiento distribuido, Hive puede utilizarse con diferentes mecanismos de almacenamiento sin detrimento de su funcionalidad. A su vez, Hadoop proporciona alta disponibilidad y métodos de gestión ante los posibles fallos del sistema lo cual da robustez a las tareas que ejecuta.

Las únicas partes que no se ejecutarán sobre Hadoop son *HiveServer2* y el *Metastore*. Esto hace que Hive cuente con múltiples instancias de estos componentes que se ejecutan de forma simultánea para que el fallo un solo componente no haga que el sistema no esté disponible. Dado que el Metastore se basa en un RDBMS para almacenar los metadatos, es necesario elegir un RDBMS de alta disponibilidad para evitar un único punto de fallo.



LLAP consiste en un demonio de larga duración que sustituye las interacciones directas con el nodo de datos de HDFS y un marco de trabajo basado en DAG estrechamente integrado. Funciones como el almacenamiento en caché, la precarga, algunos procesos de consulta y el control de acceso se trasladan al demonio. Las consultas cortas son procesadas directamente por este demonio, mientras que cualquier trabajo pesado se realizará en los contenedores estándar de YARN.

7.1.2. Base de Datos columnar

Para las consultas tradicionales de almacenamiento de datos, es bien sabido que los formatos columnares proporcionan un rendimiento superior. Hay dos formatos columnares principales en el ecosistema de big data, Apache ORC [Voh16] y Apache Parquet. Hive soporta ambos. ORC fue originalmente una parte de Hive y está estrechamente integrado en una serie de características nuevas características de Hive, como ACID y LLAP.



Apache Parquet es un formato de almacenamiento en columnas disponible para cualquier proyecto del ecosistema Hadoop, independientemente de la elección del marco de procesamiento de datos, el modelo de datos o el lenguaje de programación. 🔗 Enlace: <http://parquet.apache.org/>

Los formatos columnares suelen almacenar cada columna en un archivo separado. Sin embargo, HDFS no admite la co-ubicación de archivos relacionados; por lo tanto, esto no funciona en Hadoop. En su lugar, ORC almacena todas las columnas en un único archivo. Un archivo ORC se divide en franjas (*stripes*). Todos los valores de una columna determinada se almacenan de forma contigua dentro de una franja. Estas columnas se comprimen de forma específica para cada tipo (por ejemplo, las cadenas se codifican en forma de diccionario). La codificación de longitud se utiliza para la mayoría de los tipos.

El fichero puede comprimirse aún más utilizando técnicas estándar como zlib. La información sobre los desplazamientos de las columnas se coloca al final del archivo para que un lector pueda leer fácilmente sólo las deseadas. ORC también registra estadísticas básicas (por ejemplo, valores mínimos y máximos) por franja y grupo de filas (una colección de 10.000 filas en una franja) lo que le permite

recortar aún más los datos que lee. Por ejemplo, si una consulta solicita usuarios que viven en Albacete y, al leer un grupo de filas concreto, ORC descubre que el valor mínimo de la columna país de ese grupo de filas es Cuenca, omitirá la lectura de esas filas.

7.1.3. Particiones

Dado el gran tamaño de muchas tablas, el particionamiento de las tablas es un concepto importante en Hive. Al crearse el usuario especifica si la tabla debe ser particionada y en qué clave(s). Todo partición en Hive se basa en el valor, que cada partición contiene un único valor de la(s) clave(s) de partición. Es habitual, aunque no obligatorio utilizar un *timestamp* para el particionamiento, ya que a menudo los datos en Hadoop llegan a intervalos regulares.

La mayoría de los datos en Hive se almacenan en archivos en HDF, por lo que cada partición se almacena en un directorio separado en HDFS. Para una tabla o partición dada, normalmente habrá diferentes archivos.

7.1.4. Metadatos

Hive almacena su catálogo de datos en una base de datos relacional. Con este fin puede utilizar Oracle, SQL Server, MySQL o Postgres. Este servicio lo proporciona el módulo Metastore y se expone a través de una API de Apache Thrift para que otros sistemas y herramientas puedan acceder a los metadatos. Se eligió una base de datos relacional en lugar de almacenar los datos en Hive porque Hive no es un sistema OLTP y no está para las pequeñas lecturas y escrituras frecuentes que ejecuta el Metastore. La obtención de todos los metadatos y estadísticas necesarios para ejecutar una consulta puede tardar varios segundos cuando la consulta abarca miles de particiones. Para solucionar este problema se utiliza el almacenamiento en caché de los metadatos para que, cuando sea posible, los metadatos y las estadísticas se mantengan en la memoria.

7.1.5. Ingesta de Datos

Hive proporciona una sentencia LOAD para permitir cargar datos externos en el sistema. También admite "tablas externas", en las que los datos se escriben y son propiedad de un usuario externo a Hive. En el momento de la creación de la tabla, el usuario especifica dónde se encuentran los datos. Hive también puede escribir los datos por sí mismo mediante las sentencias INSERT o CREATE TABLE AS SELECT. Se soporta la semántica ACID transaccional estándar y UPDATE, DELETE y MERGE.

Dado que HDFS proporciona una semántica de sólo escritura, el soporte de ACID requiere que Hive almacene los registros cambiados en archivos diferentes llamados archivos delta, junto con información sobre qué transacción escribió el registro modificado. El estado de la transacción se estado de la transacción se rastrea

en el Metastore. Los lectores disponen de con una lista de transacciones abiertas y abortadas, así así como su propio ID de transacción. A partir de esto, pueden pueden determinar qué versión del registro deben leer. Cuando los archivos delta de una tabla o partición de una tabla o partición se vuelven demasiado numerosos o demasiado grandes, un proceso de compactación se ejecuta automáticamente para integrar los registros modificados en los archivos de datos base. Este proceso de compactación no no impide la lectura y escritura simultánea de los datos por parte de los usuarios.

Hive también puede ingerir datos de un flujo de entrada como Apache Kafka o Apache Flume y almacenarlos en una tabla con semántica transaccional para que los lectores puedan ver datos muy recientes.

7.1.6. SQL

El proyecto Hive se soportan todos los segmentos de ANSI SQL 2011 que son relevantes para las operaciones de almacenamiento de datos. [?]. Además de soportar los tipos de datos SQL estándar estándar, Hive también admite tipos más complejos como cadenas (cadenas de caracteres no limitadas, similares al tipo de texto RDBMS, pero almacenados en fila), listas (matrices de un tipo determinado) *structs* (un conjunto predefinido de nombres y tipos tipos de fila ANSI), y mapas (un mapeo de forma libre de cadenas de caracteres a valores). Estos tipos adicionales se utilizan con frecuencia en el mundo del Big Data, ya que no todos los datos están normalizados, especialmente cuando se ingestan por primera vez.

Function	MySQL	HiveQL
Retrieving information	SELECT from_columns FROM table WHERE conditions;	SELECT from_columns FROM table WHERE conditions;
All values	SELECT * FROM table;	SELECT * FROM table;
Some values	SELECT * FROM table WHERE rec_name = "value";	SELECT * FROM table WHERE rec_name = "value";
Multiple criteria	SELECT * FROM table WHERE rec1="value1" AND rec2="value2";	SELECT * FROM TABLE WHERE rec1 = "value1" AND rec2 = "value2";
Selecting specific columns	SELECT column_name FROM table;	SELECT column_name FROM table;
Retrieving unique output records	SELECT DISTINCT column_name FROM table;	SELECT DISTINCT column_name FROM table;
Sorting	SELECT col1, col2 FROM table ORDER BY col2;	SELECT col1, col2 FROM table ORDER BY col2;
Sorting backward	SELECT col1, col2 FROM table ORDER BY col2 DESC;	SELECT col1, col2 FROM table ORDER BY col2 DESC;
Counting rows	SELECT COUNT(*) FROM table;	SELECT COUNT(*) FROM table;
Grouping with counting	SELECT owner, COUNT(*) FROM table GROUP BY owner;	SELECT owner, COUNT(*) FROM table GROUP BY owner;
Maximum value	SELECT MAX(col_name) AS label FROM table;	SELECT MAX(col_name) AS label FROM table;
Selecting from multiple tables (Join same table using alias w/"AS")	SELECT pet.name, comment FROM pet, event WHERE pet.name = event.name;	SELECT pet.name, comment FROM pet JOIN event ON (pet.name = event.name);

Figura 7.3: HiveQL vs MySQL. Fuente:Apache

7.2. Impala

Apache Impala es un motor SQL de procesamiento masivamente paralelo (MPP) diseñado específicamente para el entorno de procesamiento de datos de Hadoop. Impala proporciona baja latencia y alta concurrencia para cargas de trabajo analíticas, funcionalidades que no se encuentran bien cubiertas por los frameworks orientados al procesamiento por lotes como MapReduce.

Impala no es un sistema monolítico, sino que se basa en la funcionalidad de los componentes comunes y compartidos del ecosistema Hadoop para proporcionar una funcionalidad similar a la de un RDBMS. En particular, Impala utiliza el *Metastore* de *Hive* como catálogo del sistema. Además, Impala no proporciona funcionalidad de persistencia por sí mismo; en cambio depende de los componentes de gestión del almacenamiento del ecosistema Hadoop (por ejemplo HDFS), que maneja a través de sus API públicas. Estos componentes de gestión del almacenamiento en efecto implementan la capa de servicio de registro de un RDBMS y definen el grado en que Impala puede soportar capacidades tradicionales de actualización y semántica transaccional

7.2.1. Visión

Impala es un motor de consulta que se integra en el entorno Hadoop y se apoya en una serie de componentes su infraestructura para ofrecer una experiencia similar a la de un RDBMS. Los formatos de archivo estandarizados y las API de los componentes que forman parte de Hadoop permiten a Impala ofrecer una funcionalidad analítica SQL muy amplia sobre una amplia variedad de formatos de datos, así como opciones de almacenamiento, como el almacenamiento de objetos en la nube como *AWS S3* y *Azure Data Lake Store*.

Sin embargo, debido a que el acoplamiento entre el motor de consulta y los gestores de almacenamiento, hay algunas diferencias notables en las áreas de la funcionalidad de actualización de SQL y la semántica transaccional. En ambos casos, Impala se basa en la funcionalidad disponible por los propios gestores de almacenamiento, pero no trata de aumentarla.

7.2.2. Arquitectura

Impala es un motor de ejecución SQL masivamente paralelo que está diseñado para ejecutarse en cientos de máquinas en clusters Hadoop. Su enfoque de la paralelización y la forma en que se implementado siguen las técnicas pioneras para los sistemas de bases de datos paralelas.

La Figura 7.4 muestra la arquitectura general de Impala.

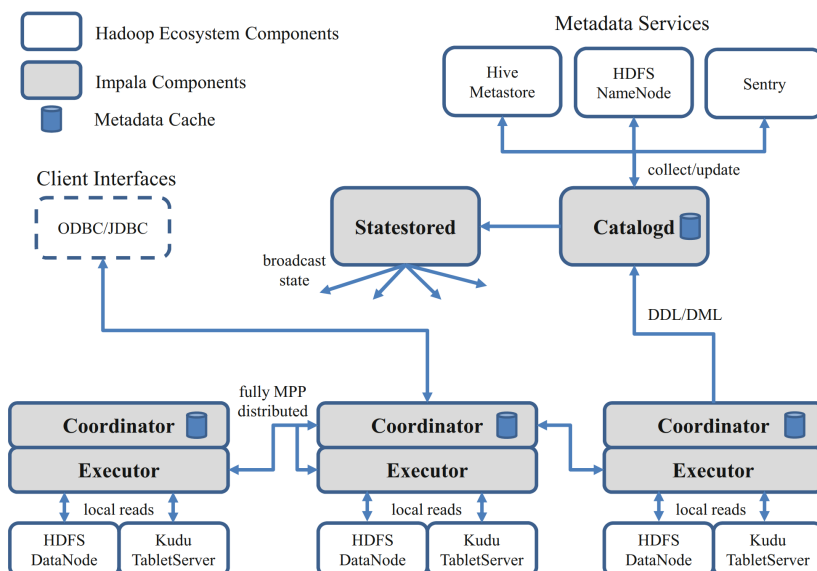


Figura 7.4: Arquitectura de Impala. Fuente: <https://impala.apache.org>

Statestored

El *Statestored* es un servicio genérico de publicación/suscripción que difunde metadatos compartidos en todo el clúster. Hay una única instancia de *statestored* para todo el clúster.

Su información se organiza en temas o *topics* que son colecciones versionadas de elementos según su contenido. Cada *impalad* se registra en el almacén de estados al inicio, momento en el que recibe una actualización masiva inicial para sincronizarlo con el estado actual de todos los *topics*. Las actualizaciones posteriores utilizan los números de versión para enviar la información de forma incremental (*deltas*).

Este daemon también envía periódicamente mensajes de aviso (*keepalive*) a todos los *impalads* registrados para detectar los fallos de los nodos y actualizar su información. También se actualiza la relación entre nodos y los diferentes *topics*, lo cual se difunde mediante el mecanismo de actualización de *topics*.

Catalogd

El *catalogd* de Impala tiene dos objetivos:

- Utiliza el *statestored* para distribuir una vista unificada de los metadatos de las tablas a los *impalads*.

- Recoge y sincroniza los metadatos de las tablas con otros componentes del ecosistema Hadoop (como el Hive Metastore y el NameNode de HDFS).

Hay una sola instancia de *catalogd* para todo el clúster.

Este servicio extrae los metadatos relevantes para la planificación y ejecución de consultas de los componentes adecuados del ecosistema Hadoop. Esos metadatos se agregan en una estructura unificada de catálogo. Cualquier cambio en ese catálogo (por ejemplo, cuando se crea una nueva tabla) se difunde a través del *statestore* y se almacenan en la caché de los demonios de Impala.

Impalad

Los daemons de Impala (*impalad*) aceptan las solicitudes de los clientes y son responsables de la planificación, coordinación y ejecución de las consultas. Almacenan en caché los metadatos recibidos del *catalogd* y son los únicos componentes que participan activamente en la ejecución de la consulta. Normalmente hay un *impalad* en cada nodo del clúster.

Este proceso asume uno o ambos de los siguientes roles:

- Coordinador de Consultas / *Query Coordinator*: acepta las solicitudes de las aplicaciones cliente (enviadas a través de Thrift), genera un plan de ejecución y luego coordina la ejecución a través de los ejecutores Impala participantes en el clúster.
- Ejecutor de Consultas / *Query Executor*: que acepta fragmentos del plan de ejecución de un coordinador y los ejecuta.



Apache Thrift es un *framework* para el desarrollo de servicios escalables que soporta diferentes lenguajes de programación. Combina una pila de software con un motor de generación de código para construir servicios que funcionen de manera eficiente. Mas info en:

No existe un proceso "maestro" que se encargue de la coordinación de las consultas de todos los clientes, por lo que los roles reales asumidos por cada proceso *impalad* desplegado son configurables, de modo que es posible escalar de forma independiente el número de coordinadores y ejecutores según los requisitos de rendimiento de la carga de trabajo.

En una instalación común de Impala, un proceso *impalad* se ejecuta en cada nodo que también ejecuta un proceso *DataNode* (para HDFS). Esto permite a Impala aprovechar la localidad de los datos y hacer una cantidad máxima de preprocesamiento local antes de enviar datos a través de la red.



Para saber más Ĩmpala: A Modern, Open-Source SQL Engine for Hadoop" Enlace: http://cidrdb.org/cidr2015/Papers/CIDR15_Paper28.pdf

7.2.3. Soporte SQL

Impala amplía su sintaxis SQL para consultar tipos complejos de forma concisa y fácil de usar. La nueva sintaxis fue diseñada para ser una extensión natural de SQL y proporcionar toda la funcionalidad de SQL en tipos complejos.

Las ideas principales se resumen en las siguientes:

- **Anidados:** Se accede a los campos anidados con la conocida notación con punto, por ejemplo, `tbl.col.nested1.nested2`.
- **Tipos Complejos:** Los tipos *map* y *array* deben ser referenciados en la cláusula *FROM* al igual que las tablas convencionales. Esto expone los campos anidados como columnas a las que se puede acceder, como es habitual, mediante expresiones habituales.
- **Colecciones:** Las colecciones pueden referenciarse mediante rutas absolutas (*FROM tbl.nested_collection*) o rutas relativas (*FROM tbl t, t.nested_collection*). La primera expone los campos de "nested_collection" pero no los de la "tbl" padre, mientras que el segundo expone tanto el padre como el hijo. Las subconsultas pueden utilizar rutas relativas correlacionadas en la cláusula *FROM* que hacen referencia a los mapas/arrays de los bloques adjuntos. Esto permite un patrón en el que los datos de las colecciones anidadas pueden ser filtrados/unidos/agregados mediante SQL.

7.2.4. Almacenamiento HDFS

HDFS es el sistema de archivos más utilizado para almacenar datos en Hadoop, ofreciendo un gran rendimiento para cargas de trabajo orientadas a la lectura. Proporciona una API de sistema de archivos con y ofrece al usuario la posibilidad de elegir entre varios formatos de archivo. Como consecuencia, para los datos residentes en HDFS, Impala soporta las sentencias SQL *SELECT* e *INSERT*; este último escribe al menos un nuevo archivo por sentencia y, por lo tanto, sólo se recomienda para la inserción masiva de datos.

Al crear una tabla en HDFS, el usuario especifica la ubicación de almacenamiento raíz de los datos físicos, así como el formato de archivo, y posiblemente una lista de columnas por las que realizar las particiones:

```
CREATE TABLE T (...)  
PARTITIONED BY (day int, month int)  
LOCATION '<hdfs-path>' STORED AS PARQUET;
```

Para una tabla no particionada, los archivos de datos se almacenan por defecto directamente en la ubicación de almacenamiento. Para una tabla particionada, los archivos de datos se colocan en subdirectorios que a su vez se encuentran en la ubicación de almacenamiento nombrada y cuyas rutas incluyen los valores de las columnas de la partición.

Por ejemplo, para el día = 17, mes = 2 de la tabla T , todos los archivos de datos se encontrarían en:

```
<hdfspath>/día = 17/mes = 2/
```

Esta forma de partición separa físicamente los datos de la tabla y permite la poda estática y dinámica de las particiones en tiempo de ejecución; sin embargo, no agrupa físicamente los archivos de datos que pertenecen a una partición individual (los bloques de esos archivos de datos están repartidos entre todos los DataNodes de HDFS).

El formato de los archivos debe elegirse teniendo en cuenta de los datos y la aplicación objetivo. Impala soporta formatos de archivo como Parquet, texto, Avro, RC y archivos de secuencia, en combinación con una serie de algoritmos de compresión (snappy, gzip, bz2, etc.). Como regla general, los formatos orientados a filas (por ejemplo, texto, Avro) favorecen las cargas de datos de datos, mientras que los formatos orientados a las columnas (p. ej, Parquet) proporcionan un rendimiento superior para consultas analíticas.

7.3. Sqoop

Sqoop es un sistema de importación y exportación de bases de datos relacionales. Fue creado por Cloudera y actualmente es un proyecto de Apache. Sqoop se utiliza para transferir datos entre almacenes de datos estructurados, como bases de datos relacionales, almacenes de datos empresariales y sistemas NoSQL y Hadoop. Extrae datos de un sistema externo a HDFS y también puede rellenar tablas en Hive y HBase. Sqoop automatiza la mayor parte de este proceso, partiendo de la información que en la base de datos origen exista para describir el esquema de los datos a importar [LL16].

Encapsula la lógica relacionada con el movimiento de los datos relacionales en Hadoop. Internamente utiliza Map/Reduce para leer/escribir datos en HDFS. Cuando se ejecuta un comando Sqoop, el conjunto de datos que necesita ser transferido se divide en varios trozos y se asigna un *mapjob* a cada trozo de datos. Estos trozos de datos se trabajan en paralelo, por lo que Sqoop es capaz de transferir datos masivos de forma eficiente.

Cuando se realiza una importación, Sqoop puede escribir en HDFS, Hive y para exportaciones puede hacer lo contrario. La importación se divide en dos actividades: conectar con la fuente de datos para recopilar algunas estadísticas, y luego disparar un trabajo MapReduce que realiza la importación propiamente dicha.

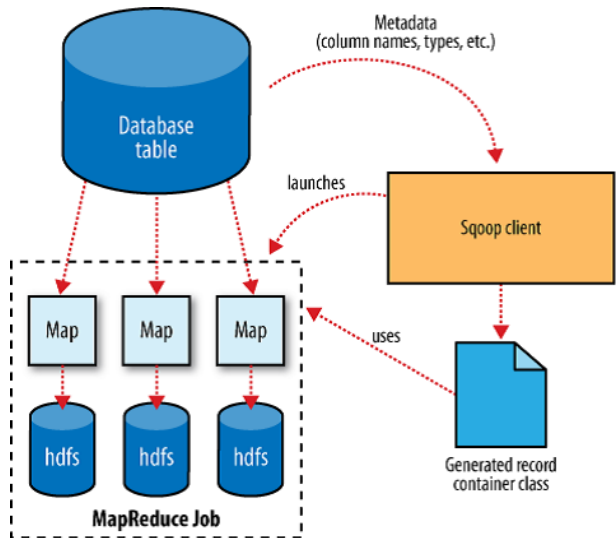


Figura 7.5: Arquitectura de Sqoop. Fuente:Apache

7.3.1. Conectores

Sqoop utiliza la noción de conectores, componentes que contienen la lógica especializada necesaria para leer y escribir en sistemas externos. Sqoop viene con dos clases de conectores: conectores comunes para lecturas y escrituras regulares y conectores rápidos que utilizan mecanismos de lotes propios de la base de datos para realizar importaciones eficientes.

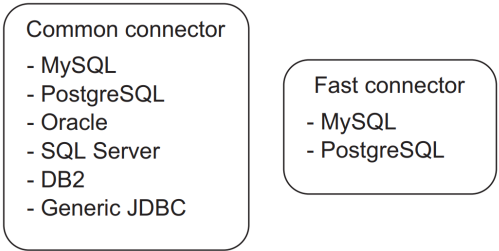


Figura 7.6: Conectores Sqoop. Fuente:Hadoop In Practice [Hol14]

7.3.2. Importación/Exportación

Sqoop importa comunmente los datos como archivos de texto separados por comas pero admite una serie de otros formatos de archivo, que pueden ser activados con los argumentos que aparecen en las siguientes opciones:

```
--as-avrodatafile # formato AVRO  
--as-sequencefile # datos en formato de ficheros de secuencias  
--as-textfile # datos en CSV
```

Sqoop se ocupa de paralelizar el trabajo de importación mediante la utilización de múltiples mapeadores (*mappers*). El primer paso de Sqoop es extraer los metadatos de la base de datos. Inspecciona la tabla que se está importando para determinar la clave primaria, y ejecuta una consulta para determinar los límites inferior y superior de los datos de la tabla. Sqoop asume una distribución algo uniforme de los datos dentro de las claves mínimas y máximas, ya que divide el rango entre las claves mínimas y máximas por el número de mapeadores. Cada mapeador recibe una consulta única que contiene un rango de la clave primaria.

Se puede configurar Sqoop para que utilice una clave no primaria con el argumento *-split-by*. Este puede ser útil en situaciones en las que la clave primaria no tiene una distribución uniforme de valores entre el mínimo y el máximo. Sin embargo, en el caso de las tablas grandes debe tener cuidado de que la columna especificada en *-split-by* esté indexada para garantizar tiempos de importación óptimos. Se puede utilizar el argumento *-boundary-query* para construir una consulta alternativa para determinar los valores mínimos y máximos.

También se pueden realizar importaciones incrementales. Sqoop soporta dos tipos: *append* funciona para datos numéricos que se incrementan con el tiempo, como las claves de autoincremento y *lastmodified* que funciona con datos con marca de tiempo. En ambos casos hay que especificar la columna mediante *-check-column*, el modo mediante el argumento *-incremental* (el valor debe ser *append* o *lastmodified*), y el valor real a utilizar para determinar los cambios incrementales mediante *-last-value*.

El Sqoop Job es una herramienta que nos permite crear y trabajar con los Saved jobs. Los Sqoop Saved jobs recuerdan los parámetros que se utilizan para especificar un job, por lo que pueden ser fácilmente reejecutados invocando el job por su handle. Un sqoop job trabajo Sqoop no es más que un registro de una sentencia de importación y/o exportación de Sqoop. En él se especifican los parámetros para identificar y recuperar el trabajo Sqoop guardado.

Esta re-ejecución o re-llamada se utiliza en la importación incremental de Sqoop, que importa las filas actualizadas de las tablas de la base de datos relacional a HDFS.

Importación a Hive

El último paso de esta técnica es utilizar Sqoop para importar los datos a una tabla Hive. La única diferencia entre una importación HDFS y una importación Hive es que la importación Hive tiene un paso de posprocesamiento en el que se crea y carga la tabla Hive. Cuando los datos se cargan en Hive desde un archivo o directorio HDFS, como en el caso de las importaciones de Sqoop, Hive mueve el directorio a su almacén en lugar de en lugar de copiar los datos en aras de la eficiencia. De esta forma, el directorio HDFS que el Sqoop MapReduce escribe no existirá después de la importación.

Lo ideal sería que una sola importación podría crear múltiples particiones de Hive. Dado que está limitado a especificar una sola clave y valor, tendría que ejecutar la importación una vez por cada valor de partición único, lo cual es laborioso. Sería mejor importar a una tabla Hive sin particiones y luego crear particiones retroactivamente en la tabla después de haberla cargado. Además, la consulta SQL que suministre a Sqoop también debe encargarse de filtrar los resultados, para que sólo se incluyan los que coincidan con la partición.

7.3.3. Comandos Sqoop

El siguiente comando importa una tabla llamada MyTbl del servidor de base de datos MySQL al directorio HDFS MyData

```
$ sqoop import \  
  --connect jdbc:mysql://localhost/userdb \  
  --username root \  
  --table MyTbl \  
  --m 1 \  
  --target-dir /MyData
```

Si la tabla MyTbl tiene una columna de fecha con marca de tiempo o fecha de última modificación (modified_date), se puede usar para obtener los datos del último día de la siguiente manera (suponiendo que está haciendo la carga incremental el 29/2/16 y quiere cargar los datos del 28/2/16):

```
$ sqoop import \  
  --connect jdbc:mysql://localhost/userdb \  
  --username Myusr \  
  --password Mypwd \  
  --table MyTbl \  
  --m 1 \  
  --target-dir /MyData/incremental_table \  
  --check-column modified_date \  
  --incremental lastmodified \  
  --last-value 2016-01-27
```

Sqoop proporciona otro modo de carga incremental, llamado `append` (en lugar de `lastmodified`), que sólo importa los registros recién añadidos (para una tabla).

```
$ sqoop import
--connect jdbc:mysql://localhost/userdb \
--username Myusr \
--password Mypwd \
--table MyTbl \
--m 1 \
--target-dir /MyData/incremental_table
--check-column modified_date
--incremental append
--last-value 9834
```

Este comando importará la tabla `TEST1` a HDFS pero también añadirá sus metadatos a Hive.

```
sqoop import \
--connect jdbc:mysql://localhost/test \
--table TEST1 \
--username root \
--m 1 \
--hive-import
```

Este comando creará una nueva tabla llamada `TEST10_ORC` en la base de datos por defecto con datos almacenados en formato de archivo ORC. En la mayoría de los casos, se almacenan los datos de la tabla Hive en formato ORC para hacer uso de las últimas optimizaciones de rendimiento, por ejemplo, la vectorización. Este comando es muy útil para crear definiciones de tablas y cargar los datos en formato ORC en un solo paso.

```
sqoop import \
--connect jdbc:mysql://localhost/test \
--table TEST10 \
--username root \
--m 1 \
--hcatalog-database default \
--hcatalog-table TEST10_ORC \
--create-hcatalog-table \
--hcatalog-storage-stanza "stored as orcfile" \
```

Bibliografía

- [Hol14] Alex Holmes. *Hadoop in practice*. Simon and Schuster, 2014.
- [LL16] Bhushan Lakhe and Lakhe. *Practical Hadoop Migration*. Springer, 2016.
- [SVGK16] Scott Shaw, Andreas François Vermeulen, Ankur Gupta, and David Kjerrumgaard. *Practical Hive: A Guide to Hadoop's Data Warehouse System*. Apress, 2016.
- [Voh16] Deepak Vohra. *Practical Hadoop ecosystem: a definitive guide to hadoop-related frameworks and tools*. Apress, 2016.